VU

7 - 15.

- Interconnection networks are classified into two categories.
  -
  -
  -
  - Static Network.
  - Dynamic Network.

- Control strategies are classified into two categories.
  -
  -
  -
  - Centralized
  - Decentralized

- Two different switching technologies are.
  -
  -
  -
  - Circuit Switching.
    - Three phases are.
      - Establish
      - Transfer
      - Disconnect
  - Packet Switching.

- Two basic types of topologies are.
  -
  -
  -
  - **Static Topology:** Ring (Loop) networks, Mesh, Torus, Tree networks, Hypercube network.
  - **Dynamic Topology:** Bus-based, Switch-based.

- Bus-based dynamic topologies are broadly classified into two categories.
  -
  -

-
- Single bus.
- Multi bus.

- In Multi Bus, following connection schemes are possible.
  -
  -
  -
  - Multiple Bus with Single Bus – Memory Connection (MBSBMC).
  - Multiple Bus with Partial Bus – Memory Connection (MBPBMC).
  - Multiple Bus with Full Bus – Memory Connection (MBFBMC).
  - Multiple Bus with Class-based Bus – Memory Connection (MBCBMC).

- The consistency models are classified into two broader categories.
  -
  -
  -
  - **Data-Centric Consistency Models:** Continuous Consistency, Consistent Ordering of Operations.
  - **Client-Centric Consistency Models:** Eventual Consistency, Monotonic Writes, Read Your Writes, Writes Follow Reads.

- Type of Mobility.
  -
  -
  -
  - **Weak mobility:** migrate only code segment.
  - **Strong mobility:** migrate execution segment and resource segment.

Process model for code migration (Fugetta et al., 98)

- Code segment: set of instructions that make up the program
- Resource segment: references to external resources
- Execution segment: store current execution state

- Types of process-to-resource binding.
  -
  -

-
- **Binding by identifier:** e.g., URL, (IPaddr:Port).
- **Binding by type:** e.g., monitor, printer.
- **Binding by value:** e.g., standard libraries.

- Type of resources.
  -
  -
  -
  - **Unattached resources:** Can be easily moved (e.g., data files).
  - **Fastened resources:** Can be used but at a high cost (e.g., local databases, web sites).
  - **Fixed resources:** Cannot be moved (e.g., local devices).

- **[Migrating the Entire Memory Image]** There are, in principle, three ways to handle migration.
  -
  -
  -
  - **Send Memory to New Machine and Update Changes:** Pushing memory pages to the new machine and resending the ones that are later modified during the migration process.
  - **Move Only Needed Memory While Running:** Letting the new virtual machine pull in new pages as needed, that is, let processes start on the new virtual machine immediately and copy memory pages on demand.
  - **Stop, Move Everything, and Start Again:** Stopping the current virtual machine; migrate memory and start the new virtual machine.

- There are two main types of message passing.
  -
  -
  -
  - Synchronous.
  - Asynchronous.

- **[Message Passing Programming Modes]** Different communication modes that can be used for message passing programming are.
  -
  -
  -
  - **Blocking / Non-blocking.**

- A routine is blocking if its completion (return of control to the calling routine) may depend on an external event (an event that is outside the control of the routine itself). Example: A send is blocking if it does not return until there is a matching receive.
- A routine is non-blocking if it is guaranteed to complete regardless of external events (e.g., the other processors). Example: A send is non-blocking if it is guaranteed to return whether or not there is a matching receive.

➢ **Synchronous / Asynchronous.**
- A routine is synchronous if its operation happens within the flow of control of the calling process.
- A routine is asynchronous if it initiates an operation that happens logically outside the flow of control of the calling process. The important practical distinction is whether the program may be required to check for completion of the operation before proceeding.

➢ **Buffered / Unbuffered.**
- A routine that uses buffered message passing is a program that sends and receives messages using a buffer. In this mode, messages are queued in the buffer until they are ready to be sent or received.
- In unbuffered, a routine may be used to perform a specific message-passing task, such as sending or receiving a message between two processes. Does not use a buffer. Messages are sent and received immediately without any buffering. This mode is useful when low latency is required.

- An asynchronous communication completes before the message is received. It has three modes.
  - ➢
  - ➢
  - ➢
  - ➢ **Ready send:** completes immediately, if the receiver is ready for the message it will get it, otherwise the message is dropped silently.
  - ➢ **Buffered send:** completes immediately, if receiver not ready, MPI buffers the message locally.
  - ➢ **Standard send:** completes once the message has been sent, which may or may not imply that the message has arrived at its destination.

- Benefits of the Message Passing Interface (MPI).
  - ➢
  - ➢
  - ➢
  - ➢ **Availability:** A variety of implementations are available, both vendor and public domain.
  - ➢ **Functionality:** Over 115 routines are defined in MPI-1 alone.
  - ➢ **Scalability:** MPI is designed to scale to large numbers of processors, which makes it well-suited for high-performance computing.

- **Standardization:** MPI is the only message passing library which can be considered a standard. It is supported on virtually all HPC platforms. Practically, it has replaced all previous message passing libraries.
- **Portability:** There is no need to modify your source code when you port your application to a different platform that supports (and is compliant with) the MPI standard.

- Thread Models.
  -
  -
  -
  - One to One.
  - Many to One.
  - Many to Many.

- Why Threads? (Characteristics of Threads).
  -
  -
  -
  - **Scheduling and Load Balancing:** Threads can be managed by the operating system or libraries to ensure the work is shared fairly across processors, so no processor stays idle while others are overworked.
  - **Ease of Programming:** Using threads simplifies breaking a task into smaller parts that can run at the same time, helping programmers create faster and more efficient programs.
  - **Widespread Use:** Threads are commonly used in many applications, from web browsers to games, because they help handle multiple tasks (like downloading and playing videos) smoothly.
  - **Latency hiding:** Threads can perform other tasks while waiting for something slow (like reading from a disk), reducing delays and making the program seem faster.
  - **Portability:** Thread-based programs can work on many types of systems without needing big changes, as threads are supported by most operating systems and programming languages.

- Mutex-locks have two states.
  -
  -
  -
  - Locked.
  - Unlocked.

- Types of mutex.
  -
  -
  -
  - A normal mutex.
  - A recursive mutex.
  - Error check mutex.

- Types of tasks in term of interdependency.
  -
  -
  -
  - Dependent tasks.
  - Independent tasks.

- Decomposition Techniques.
  -
  -
  -
  - Data Decomposition.
  - Exploratory Decomposition.
  - Speculative Decomposition.
  - Recursive Decomposition.

- Data Decomposition Steps.
  1.
  2.
  3.
  4. **Data Splits into Parts:** The data on which the computations are performed is partitioned.
  5. **Tasks Are Created From Data Parts:** This data partitioning is used to induce a partitioning of the computations into tasks.

- In Speculative Decomposition, there are generally two approaches.
  -
  -
  -
  - **Conservative approaches:** May yield little concurrency.
  - **Optimistic approaches:** May require roll-back mechanism in the case of an error.

- There are three main reason for I/O bottleneck.
  -
  -
  -
  - Increase in number of CPUs.
  - Increasing CPU Speed as compared to I/O.
  - New application domains that increasing I/O demand.

- MPI-IO can be done in 2 basic ways.
  -
  -
  -
  - **Collective MPI-IO:** All processes work together to read or write data to a file. They coordinate with each other to perform the I/O operation, making it efficient.
  - **Independent MPI-IO:** Each process performs its own I/O operation without coordinating with other processes. Each process reads or writes its own portion of the data to the file.

- Buffered I/O.
  -
  -
  -
  - Advantages.
    - Aggregates smaller read/write operations into larger operations.
  - Disadvantages.
    - Requires additional memory for the buffer.

- Scaling Characteristics of Parallel Programs.
  -
  -
  -
  - We call such systems scalable parallel systems.
  - Scalability and cost-optimality are therefore related.
  - In many cases, T_o grows sub-linearly with respect to T_s.
  - A scalable parallel system can always be made cost-optimal if the number of processing elements and the size of the computation are chosen appropriately.
  - Recall that cost-optimal parallel systems have an efficiency of $\Theta(1)$.

- MapReduce has three main phases.
  -
  -
  -
  - Map.
  - Sort.
  - Reduce.

- Key functions of Hadoop.
  -
  -
  -
  - Storing and processing large amounts of data across a cluster of computers.
  - Management of the cluster.

- Hadoop Goods & Bads.
  -
  -
  -
  - Good For.
    - Repetitive tasks on big size data.
  - No Good For.
    - Complex processing requiring various phases and/or iterations.
    - Processing small to medium size data.
    - Replacing a RDMBS.

- In Google File System (GFS) Architecture, Master stores three types.
  -
  -
  -
  - File & chunk namespaces.
  - Mapping from files to chunks.
  - Location of chunk replicas.

- Google File System (GFS) limitations.
  -
  -
  -
  - Custom designed.
  - Only viable in a specific environment.

- Limited security.

- Hadoop Distributed File System (HDFS) Basic Features.
  -
  -
  -
  - Streaming access to file system data.
  - Can be built out of commodity (item) hardware.
  - High throughput.
  - Highly fault-tolerant.
  - Suitable for applications with large data sets.

- Hadoop Distributed File System (HDFS) clusters use 2 types of nodes.
  -
  -
  -
  - **Namenode (master node):** It's like the "brain" of the HDFS cluster. It keeps track of where all the data is stored and manages the entire system. It's responsible for maintaining a directory hierarchy of the data.
  - **Datanode (worker node):** It's like the "storage worker" of the HDFS cluster. It stores actual data in the form of blocks and replicates them for backup. Datanodes work together to provide access to the stored data.

- Difference between Local Area Network (LAN) & Wide Area Network (WAN).
- .
- .
- .

Local Area Network (LAN)	Wide Area Network (WAN)
Local area network (LAN) – connects nodes in single building, fast & reliable.	Wide area network (WAN) – connects nodes across large geographic area.
Media: twisted-pair, coax, fiber.	Media: fiber, microwave links, satellite channels.
Bandwidth: 10-100MB/s	Bandwidth: 1.544MB/s (T1), 45 MB/s (T3)

- Applications of GPU.
  -
  -
  -
  - **Data Analysis:** Databases, matrix algebra, sorting and searching, data mining.
  - **Image Processing:** Registration, interpolation, feature detection, recognition, filtering.

- **Scientific Computing:** CFD, Molecular Dynamics, Physical modeling, computational engineering, Genome Sequencing, Mechanical Simulation, Quantum Electrodynamics, Game effects (FX) physics.

- Processors use message-passing libraries for communication among themselves. Along with the data being sent, the message contains the following components.
  -
  -
  -
  - The address of the processor from which the message is being sent.
  - Starting address of the memory location of the data in the sending processor.
  - Data type of the sending data.
  - Data size of the sending data.
  - The address of the processor to which the message is being sent.

- The message passing model demonstrates the following characteristics.
  -
  -
  -
  - **Tasks Work Separately but Can Be on Many Computers:** A set of tasks that use their own local memory during computation. Multiple tasks can reside on the same physical machine as well across an arbitrary number of machines.
  - **Tasks Share Information by Sending Messages:** Tasks exchange data through communications by sending and receiving messages.
  - **Both Sides Must Work Together to Share Data:** Data transfer usually requires cooperative operations to be performed by each process. For example, a send operation must have a matching receive operation.

- Types of Parallel Computing Models.
  -
  -
  -
  - Single Instruction Multiple Data (SIMD).
  - Multiple Instruction Multiple Data (MIMD).
  - Single Program Multiple Data (SPMD).

- A message-passing library specifications.
  -
  -
  -

- Not a language or compiler specification.
- Not a specific implementation or product.
- Extended message-passing model.

- Many parallel programs can be written using just these six functions, only two of which are non-trivial.
  -
  -
  -
  - MPI_INIT
  - **MPI_SEND**
  - MPI_COMM_SIZE
  - MPI_COMM_RANK
  - **MPI_RECV**
  - MPI_FINALIZE

- Key steps in design of Parallel Algorithms.
  -
  -
  -
  - Dividing a computation into smaller computations.
  - Assigning them to different processors for parallel execution.

- Collections of system software and libraries have grownup to address I/O issues.
  -
  -
  -
  - MPI-IO.
  - At Parallel file systems.
  - High level libraries.

- Advantages & Disadvantages of Striping.
  -
  -
  -
  - Advantages.
    - An increase in the bandwidth available when accessing the file.
    - An increase in the available disk space for storing the file.
  - Disadvantages.
    - Increased overhead due to network operations and server contention.

- I/O Best Practices.
  -
  -
  -
  - Avoid opening and closing files frequently.
  - Place small files on single OSTs
  - Small files (< 1 MB to 1 GB) accessed by a single process.
  - Medium sized files (> 1 GB) accessed by a single process.
  - Large files (>> 1 GB).

- Namenode failure prevention schemes.
  -
  -
  -
  - Namenode File Backup.
  - Secondary Namenode.

- GPU Accelerated Libraries.
  -
  -
  -
  - OpenCV.
  - NVIDIA cuBLAS.
  - NVIDIA cuRAND
  - NVIDIA cuFFT
  - NVIDIA NPP

- Bill Dally of Stanford University considers power and massive parallelism as the major benefits of GPUs over CPUs for the future. Two Aspects,
  -
  -
  -
  - Data Access Rate Capability.
    - Bandwidth.
  - Data Processing Capability.
    - How many operations (ops) per sec.